Implementation Document

for

Credit Snap

Version 1.0

Prepared by

Group 6 : **Group Name:** ***Bug Dealers***

Kasavajjala Sai Shreyas	**240530**	ksshreyas24@iitk.ac.in
Gade V S S R K Tejas Reddy	**240390**	tejasreddy24@iitk.ac.in
Palagiri Sathish Reddy	**240717**	psreddy24@iitk.ac.in
Kondreddy Sai Haneesh Reddy	**240551**	haneesh24@iitk.ac.in
Yerraiahgari Ram Charan Goud	**241212**	ramcharang24@iitk.ac.in
Golla Lekha Harshaa	**240402**	lekhahg24@iitk.ac.in
Yash Raj	**241202**	yashr24@iitk.ac.in
Korada Jayavardhan	**240554**	jkorada24@iitk.ac.in
Boppudi Sai Chaitanya	**240283**	boppudis24@iitk.ac.in
Panchaneni Ashwin Rao	**240727**	ashwinr24@iitk.ac.in

Course: **CS253**

Mentor TA: **George T L**

Date: **March 27, 2026**

CONTENTS

CONTENTS ii
REVISIONS iii

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 6

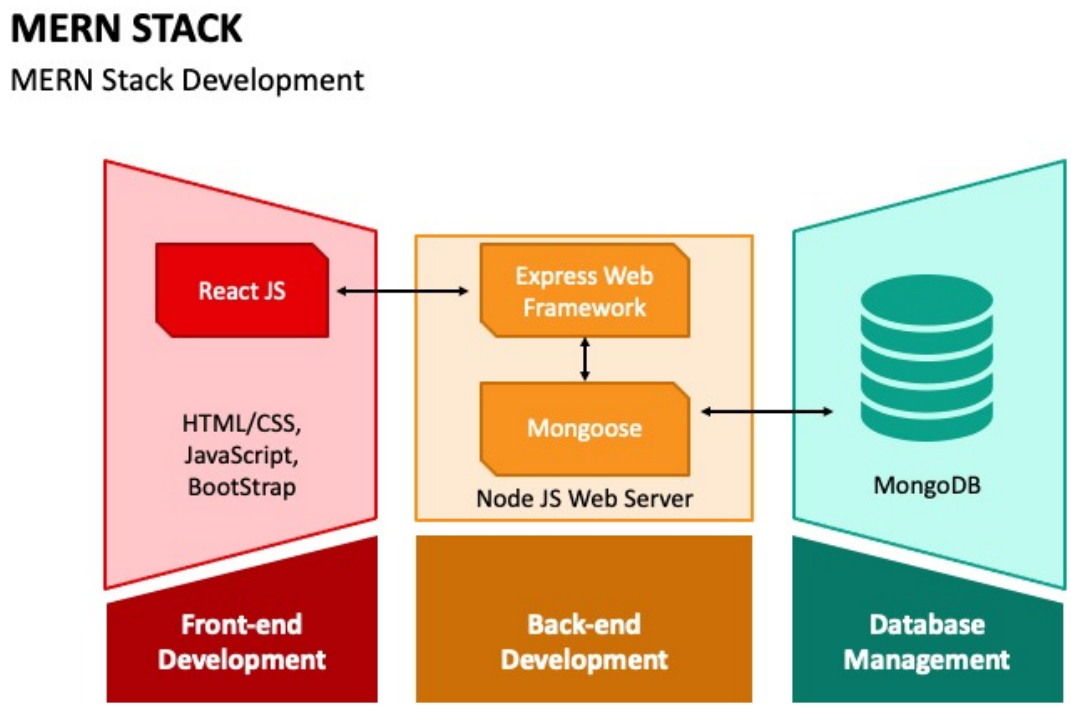
3 COMPLETENESS 13

APPENDIX A - GROUP LOG 17

REVISIONS

Version	Primary Author(s)	Description of Version	Date Completed
v1.0	Kasavajjala Sai Shreyas Gade V S S R K Tejas Reddy Palagiri Sathish Reddy Kondreddy Sai Haneesh Reddy Yerraiahgari Ram Charan Goud Golla Lekha Harshaa Yash Raj Korada Jayavardhan Boppudi Sai Chaitanya Panchaneni Ashwin Rao	The first version of the Implementation Document	27/03/26

1 Implementation Details



CreditSnap is a comprehensive web-based campus dining and debt management platform built using the **MERN** stack, which leverages MongoDB, Express.js, React.js, and Node.js. This technology stack was chosen for its ability to seamlessly handle the real-time ordering and complex financial tracking requirements of our platform.

MongoDB, a NoSQL database, provides a flexible and scalable data storage solution, ideal for managing diverse and dynamic data such as student profiles, live canteen menus, and continuous transaction records efficiently.

Express.js, a lightweight and flexible web application framework for Node.js, simplifies backend development by offering robust routing and secure middleware capabilities, which are essential for processing our RESTful APIs and order logic.

React.js, a powerful JavaScript library, enables us to create dynamic and interactive user interfaces, ensuring a seamless and responsive experience across both the student ordering portals and the canteen owner dashboards.

Node.js, with its event-driven architecture, allows for non-blocking I/O operations, ensuring high performance and scalability for our web application, especially when handling simultaneous concurrent orders from students across the campus.

Programming Language, Framework, and Libraries

Programming Language:

CreditSnap has been developed primarily using the JavaScript programming language. CreditSnap follows the model-view-controller (MVC) architectural pattern.

For the **Backend**, we have used:

1. **JavaScript (Node.js):** The benefit of using **JavaScript** for the backend is that, as compared to traditional languages like C++/Java, it offers much more flexibility by providing a wide range of frameworks enabling efficient asynchronous operations. This is crucial for handling multiple concurrent student food orders. It also allows code reusability across the MERN stack and ensures scalability with its robust ecosystem of tools and resources.

For the **Frontend**, we have used:

1. **HTML:** HTML is used for structuring web pages. Every modern browser supports HTML, and it provides a reliable, easy-to-use foundation for building our student and owner dashboards.

2. **CSS (Tailwind CSS):** Cascading Style Sheets (CSS), specifically implemented via the Tailwind CSS framework, are employed for styling HTML elements, providing a consistent, highly responsive, and visually appealing layout across multiple web pages. It is also highly maintainable; utility classes allow for rapid UI development, meaning global design changes can be made quickly without writing bulky custom CSS files.

3. **JS (JavaScript / React.js):** JavaScript plays a dual role here. While Node.js handles the backend, JavaScript on the front end—powered by the **React.js** library—adds high interactivity to the user interface and makes the application dynamic for the user (e.g., live cart updates and instant open/close status toggles). JavaScript is very fast because it can be run immediately within the client-side browser. Unless outside resources are required, JavaScript is unhindered by network calls to a backend server.

For the **DBMS** (Database Management System), we have used:

1. **MongoDB:** The reason behind this is that **MongoDB** is highly flexible and faster than traditional relational databases like MySQL when handling large amounts of dynamic or semi-structured data. This makes it perfect for storing the diverse information managed by CreditSnap, such as dynamic canteen menus, user authentication profiles, and continuous debt transaction logs.

Frameworks:

Build System: We have used **Node.js** and **NPM** due to the following benefits:

1. **NPM (Node Package Manager):** Node.js comes with a built-in package manager called npm, which is the largest ecosystem of open-source libraries in the world.

2. Npm allows developers to easily install, manage, and share reusable code packages, making it quick and convenient to add functionality (such as Axios for API calls and Tailwind for styling) to our applications.

3. It provides numerous libraries and reusable templates that significantly accelerate the development timeline.

Node.js: We have used **Node.js** as our backend framework/runtime, which has the following features:

1. Node.js is a powerful JavaScript runtime environment that allows developers to build scalable and high-performance applications.

2. Node.js is designed to handle asynchronous I/O operations efficiently. It uses an event-driven, non-blocking I/O model, which allows multiple operations to be performed concurrently without blocking the execution thread. This makes Node.js well-suited for handling high-throughput applications, such as processing simultaneous food orders and real-time debt transactions for CreditSnap.

3. Node.js is cross-platform, meaning it runs on various operating systems such as Windows, macOS, and Linux. This allows developers to write applications once and deploy them across different platforms without modification, increasing development efficiency and flexibility.

4. Node.js provides support for streaming data, allowing developers to process large files or data streams incrementally without loading the entire dataset into memory. This is particularly useful for handling file I/O, network communication, and real-time data processing between the student and canteen owner dashboards.

5. Node.js facilitates seamless integration with cloud platforms, enabling effortless deployment, scaling, and management of applications in distributed environments.

Vite: We have used **Vite** as our frontend build tool, offering the following advantages:

1. Vite is a modern build tool that makes developing our React frontend significantly faster and more efficient compared to older, heavier tools.

2. It allows our local development server to start up almost instantly. Instead of processing the entire project before starting, it only loads the exact files we are currently working on, drastically reducing wait times.

3. It automatically and instantly updates the website in the browser the moment we save a file. This means we can tweak the user interface and see the changes immediately without having to manually refresh the page or lose the data we were testing.

Libraries:

We have used the **Express** and **React.js** libraries for our project due to the following benefits:

EXPRESS

1. The Express library is a popular web application framework for Node.js.
2. It simplifies the process of building web applications and RESTful APIs in Node.js by providing a minimalistic and flexible set of features, which is essential for smoothly managing our canteen menus and debt records.
3. It allows developers to use middleware functions to perform tasks such as parsing request bodies, handling sessions, secure user authentication (protecting owner versus student routes), and logging. Middleware functions can be chained together to handle requests in a modular and reusable way.
4. It provides a simple and intuitive way to define routes for handling different HTTP requests (GET, POST, PUT, DELETE, etc.) on various URLs, enabling seamless menu editing and order placements.
5. Express simplifies error handling by providing middleware that can catch and process errors, making it easier to manage and debug the application during complex transaction flows.

REACT.JS

1. React.js is a popular JavaScript library for building user interfaces, particularly for web applications. Developed by Facebook, React.js focuses on component-based architecture and provides a declarative syntax for creating highly interactive UIs, such as our dynamic student ordering pages and canteen owner dashboards.
2. React.js utilizes a virtual DOM to improve performance. Instead of directly manipulating the browser's DOM, React.js works with a lightweight representation called the virtual DOM. React then compares the virtual DOM with the actual DOM and only updates the parts that have changed. This results in faster rendering, which is crucial when rapidly updating active orders or toggling real-time canteen availability statuses.
3. React.js follows a unidirectional data flow pattern, where data flows down from parent components to child components via props. This helps in maintaining a clear and predictable data flow throughout the application, ensuring that complex states like shopping carts and debt summaries remain perfectly synchronized without bugs.

SOCKET.IO

1. Socket.IO is a powerful JavaScript library that enables real-time, bidirectional communication between the client (frontend) and the server (backend). Unlike traditional HTTP-based communication, which follows a request-response model, Socket.IO allows continuous data exchange, making it ideal for applications that require instant updates.

2. In CreditSnap, Socket.IO is used to implement real-time features such as live order updates, instant notifications, and dynamic synchronization between students and canteen owners. For example, when a student places an order, the canteen owner dashboard is updated immediately without requiring a page refresh. Similarly, any changes in order status (accepted, rejected, or completed) are instantly reflected on the student side.

3. Socket.IO works on top of WebSockets but also provides fallback mechanisms (such as long polling) to ensure reliable communication across different browsers and network conditions. This guarantees that real-time features remain functional even in less optimal environments.

4. On the frontend, a dedicated `socket.js` module manages the connection with the backend server and listens for specific events such as order updates, payment confirmations, and notifications. On the backend, Socket.IO is integrated with the Node.js server to emit and handle events corresponding to different actions performed in the system.

5. The use of Socket.IO significantly enhances user experience by reducing latency and eliminating the need for repeated API polling. It ensures that both students and canteen owners are always viewing the most up-to-date information, which is crucial for efficient order handling and accurate financial tracking in a real-time campus environment.

RAZORPAY

1. Razorpay is a robust payment gateway library used to handle secure digital transactions within our application.

2. It enables students to easily settle their canteen debts through direct online payments using various methods, directly fulfilling the ”Pay Later” clearance feature defined in our SRS.

3. The library provides a highly secure, reliable, and easily integrated API, ensuring that sensitive financial data is processed safely and efficiently between the student, the platform, and the banking network without storing payment details on our local servers.

2 Codebase

Github Repository - CreditSnap

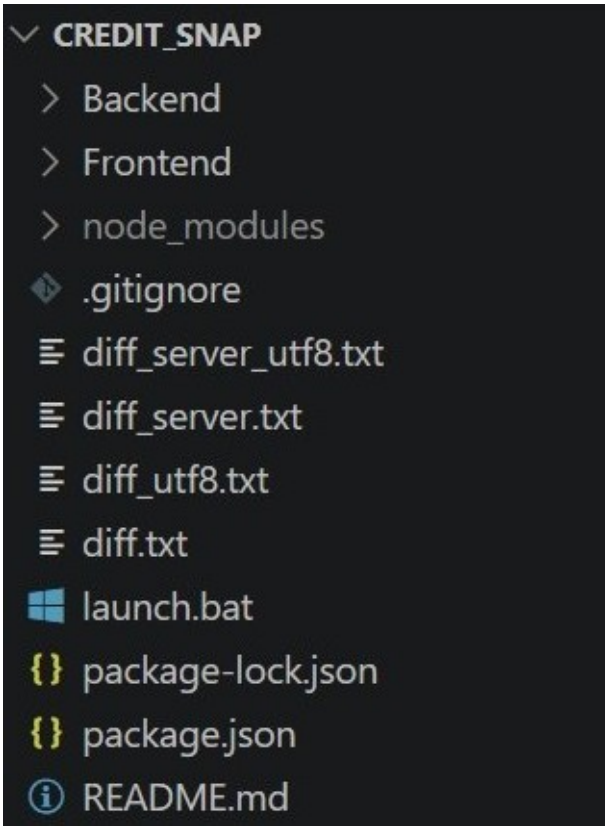
Code Structure

The above GitHub link contains all the source code of our web application.

The repository is divided into two main parts:

- Frontend
- Backend

The following images show the overall structure of the CreditSnap repository:

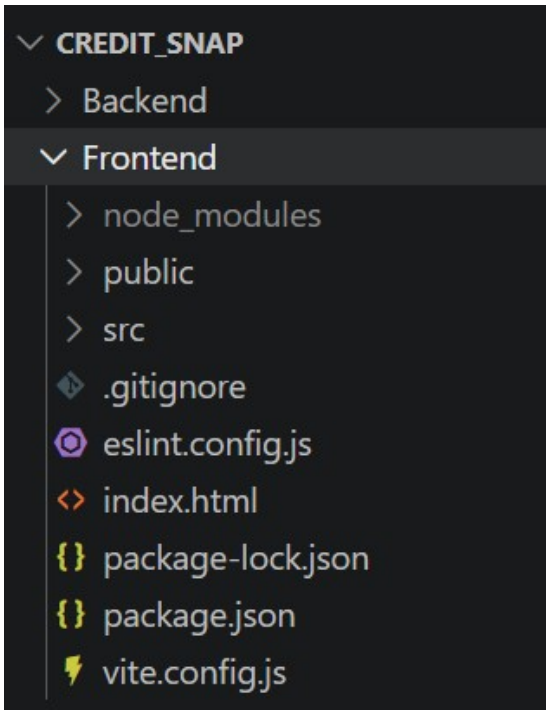


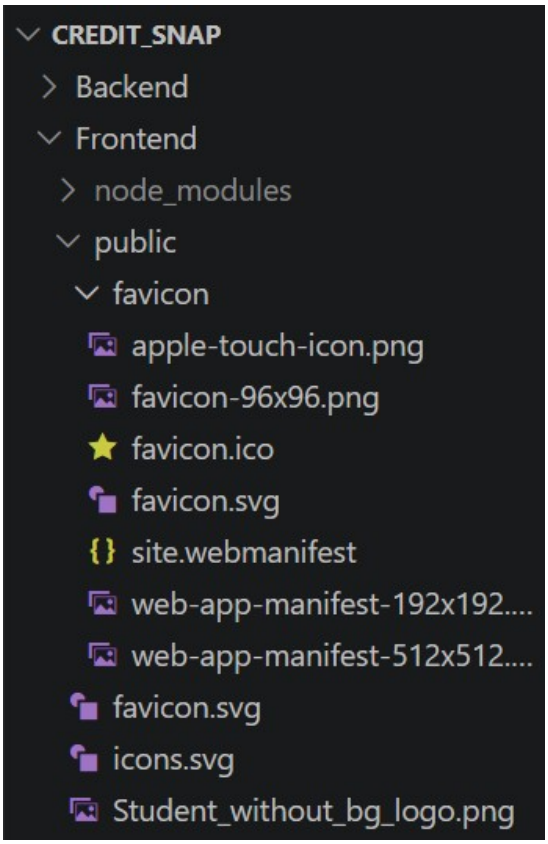
Frontend Folder Structure Overview:

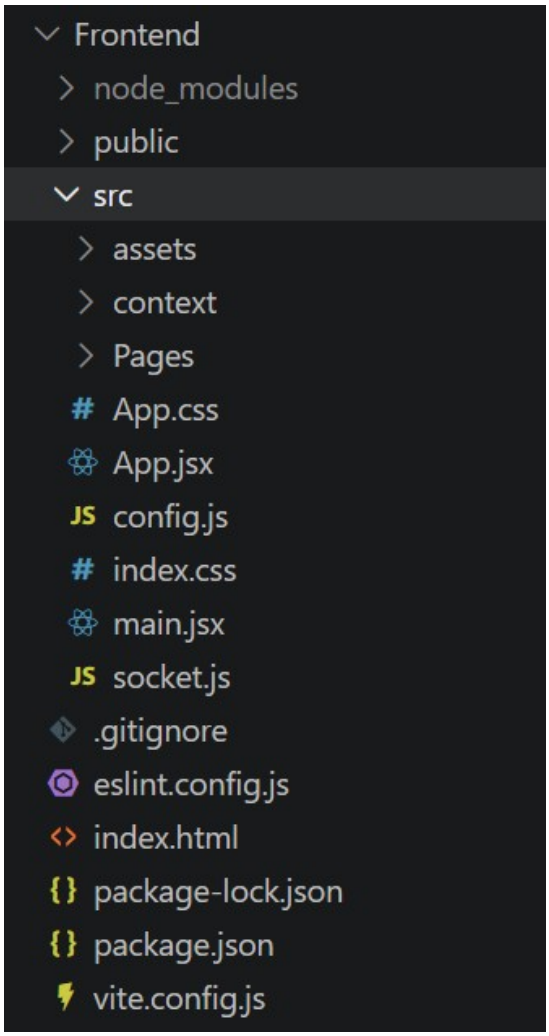
The frontend folder contains all the necessary files for the React-based client-side application, managing UI components, routing, and API interactions. Below is a breakdown of its structure:

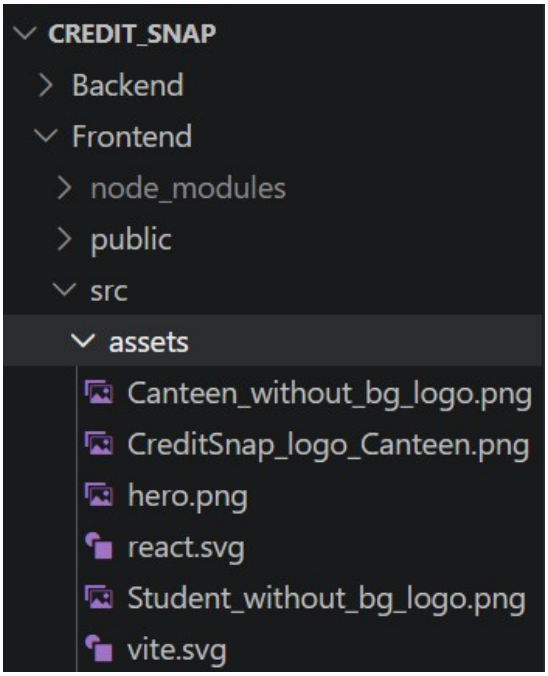
- **index.html -** Main HTML file. The root HTML file serves as the entry point for the React app. It contains the `<div id="root"></div>` where React components are mounted and typically includes script.

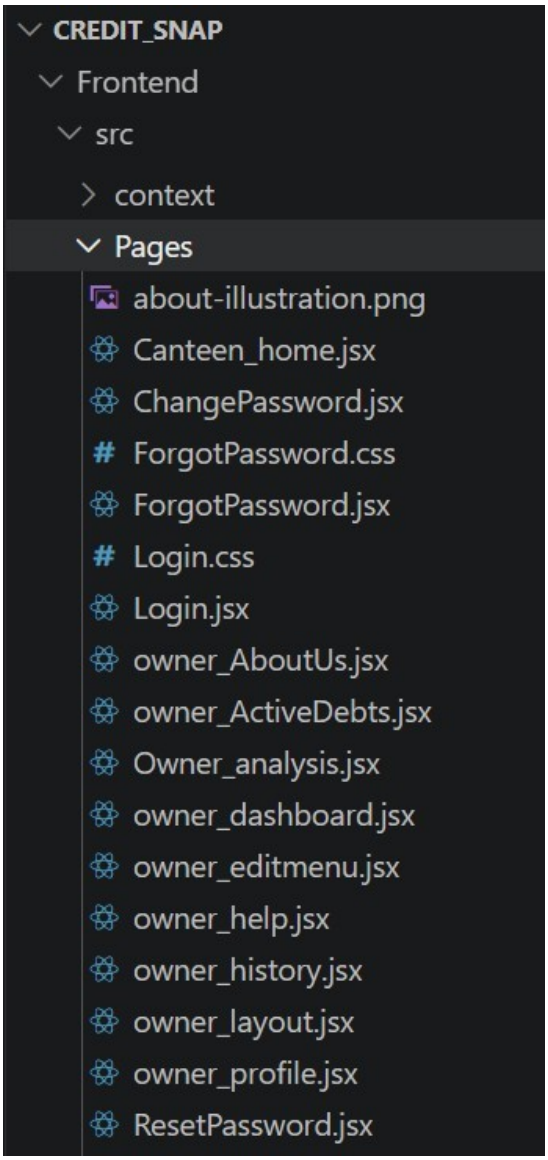
- **src/ -** Main Source Code Directory This directory contains all the source code and logic for the React application links for loading the React application.
  - **assets/** – Static Files
    Stores static assets such as images and icons required for the UI. These files are typically imported into components when needed.
  - **context/** – React Context for State Management
    Contains React Context API implementations that manage global state across the application. Includes `NotificationContext.jsx` for handling real-time notification state throughout the app.
  - **Pages/** – Page Components for Routing
    Stores full-page components used in React Router for navigation. This directory is divided into:
    - **Authentication Pages:** Examples include Login, Signup, Forgot password etc.
    - **Student Pages:** Examples include student dashboard, canteens, view debts etc.
    - **Owner (Canteen) Pages:** Examples include owner dashboard, active debts, menu etc.
  - **App.css & index.css** – Global Stylesheets
    - **App.css** – Contains styles specific to the `App.jsx` component.
    - **index.css** – Defines global styles applied across the entire application.
  - **App.jsx** – Main Application Component
    The top-level React component that renders the app's structure, including routing, layout, and context providers. It acts as the root component that encapsulates all other components.
  - **config.js** – Static Configuration Values
    Stores predefined constants such as API endpoints, theme settings, and default configuration data used throughout the app.
  - **socket.js** – WebSocket Integration
    Manages real-time WebSocket connections for live communication between the client and the backend server.
  - **main.jsx** – React App Entry Point
    The main file responsible for rendering the React app inside `index.html`. It includes React's `createRoot` function and wraps the application with providers.
- **public/ – Public Assets Directory**
  Contains publicly accessible static image files which includes CreditSnap logo for student account.

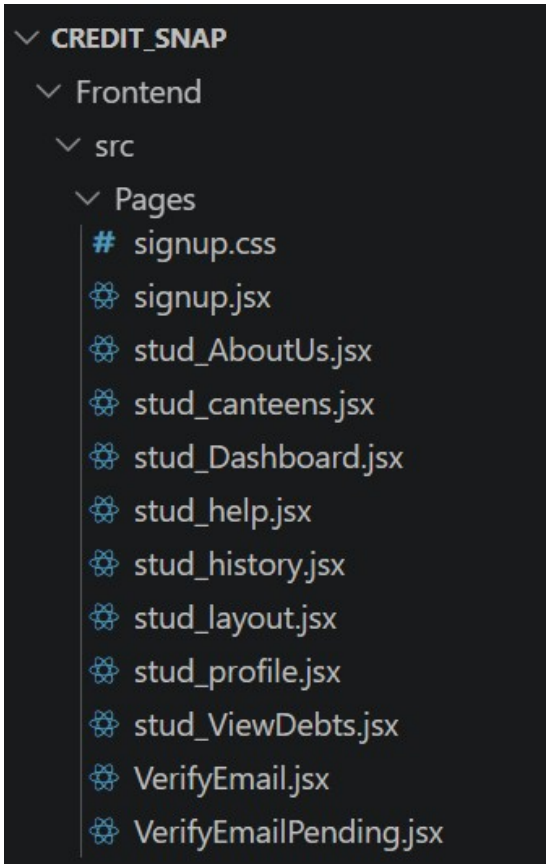

CREDIT_SNAP
Backend
Frontend
node_modules
public
src
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
vite.config.js

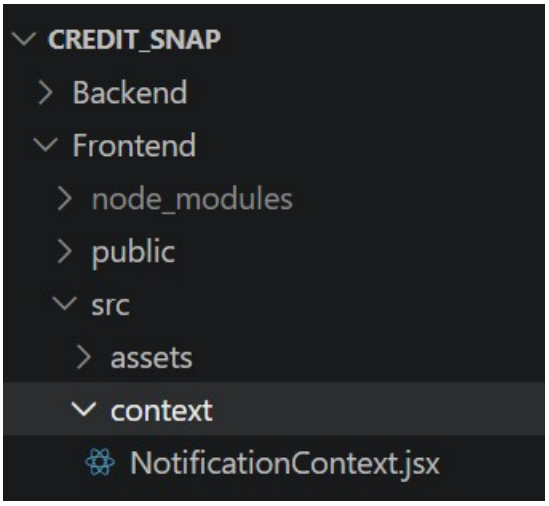

CREDIT_SNAP
Backend
Frontend
node_modules
public
favicon
apple-touch-icon.png
favicon-96x96.png
favicon.ico
favicon.svg
site.webmanifest
web-app-manifest-192x192....
web-app-manifest-512x512....
favicon.svg
icons.svg
Student_without_bg_logo.png


Frontend
node_modules
public
src
assets
context
Pages
App.css
App.jsx
config.js
index.css
main.jsx
socket.js
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
vite.config.js


CREDIT_SNAP
Backend
Frontend
node_modules
public
src
assets
Canteen_without_bg_logo.png
CreditSnap_logo_Canteen.png
hero.png
react.svg
Student_without_bg_logo.png
vite.svg


CREDIT_SNAP
Frontend
src
context
Pages
about-illustration.png
Canteen_home.jsx
ChangePassword.jsx
ForgotPassword.css
ForgotPassword.jsx
Login.css
Login.jsx
owner_AboutUs.jsx
owner_ActiveDebts.jsx
Owner_analysis.jsx
owner_dashboard.jsx
owner_editmenu.jsx
owner_help.jsx
owner_history.jsx
owner_layout.jsx
owner_profile.jsx
ResetPassword.jsx


CREDIT_SNAP
Frontend
src
Pages
signup.css
signup.jsx
stud_AboutUs.jsx
stud_canteens.jsx
stud_Dashboard.jsx
stud_help.jsx
stud_history.jsx
stud_layout.jsx
stud_profile.jsx
stud_ViewDebts.jsx
VerifyEmail.jsx
VerifyEmailPending.jsx


CREDIT_SNAP
Backend
Frontend
node_modules
public
src
assets
context
NotificationContext.jsx

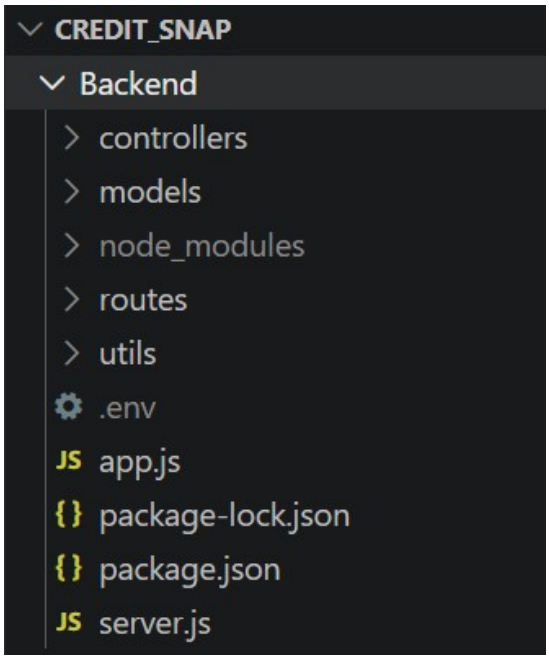
Backend Folder Structure Overview:

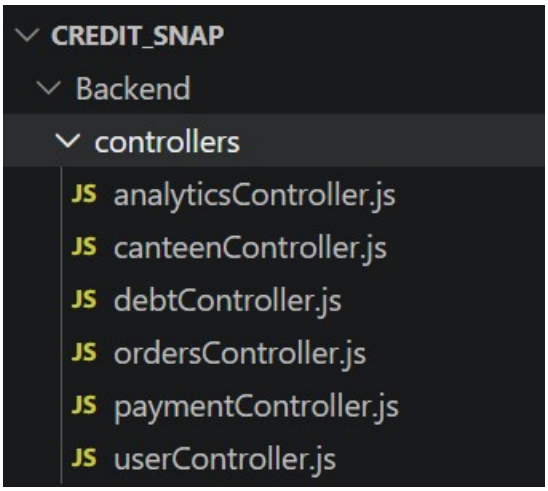
Backend/ – Server-Side Application Directory

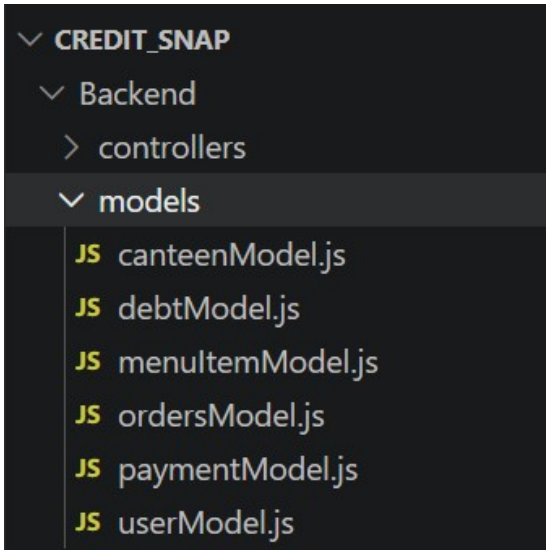
This directory contains all the server-side logic, database schemas, API routes, and utility functions for the Node.js/Express backend.

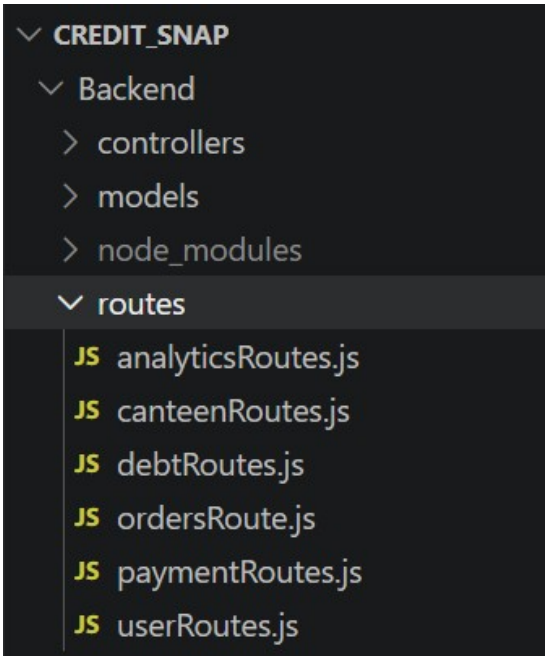
- **controllers**/ – Business Logic Layer
  Handles the core logic for different features of the CreditSnap application. Each controller corresponds to a specific domain:
  - **analyticsController.js** – Handles analytics data processing and reporting logic.
  - **canteenController.js** – Manages canteen-related operations such as registration and menu management.
  - **debtController.js** – Controls debt creation, tracking, and settlement logic.
  - **ordersController.js** – Handles order placement, updates, and retrieval operations.
  - **paymentController.js** – Manages payment processing and transaction handling.
  - **userController.js** – Handles user authentication, registration, and profile management.
- **models**/ – Database Schemas
  Defines the structure of database collections using MongoDB/Mongoose. Ensures data consistency across the application:
  - **canteenModel.js** – Schema for canteen profiles and related data.
  - **debtModel.js** – Schema for debt records between students and canteens.
  - **menuItemModel.js** – Schema for individual canteen menu items.
  - **ordersModel.js** – Schema for order records and their statuses.
  - **paymentModel.js** – Schema for payment transactions.
  - **userModel.js** – Schema for student user accounts and credentials.
- **routes**/ – API Endpoints
  Defines the backend's API routes, linking HTTP requests to their respective controllers. Each route file corresponds to a specific feature:
  - **analyticsRoutes.js** – Routes for analytics and reporting endpoints.
  - **canteenRoutes.js** – Routes for canteen management operations.
  - **debtRoutes.js** – Routes for debt creation and management.
  - **ordersRoute.js** – Routes for order processing endpoints.
  - **paymentRoutes.js** – Routes for payment-related API calls.
  - **userRoutes.js** – Routes for user authentication and profile operations.

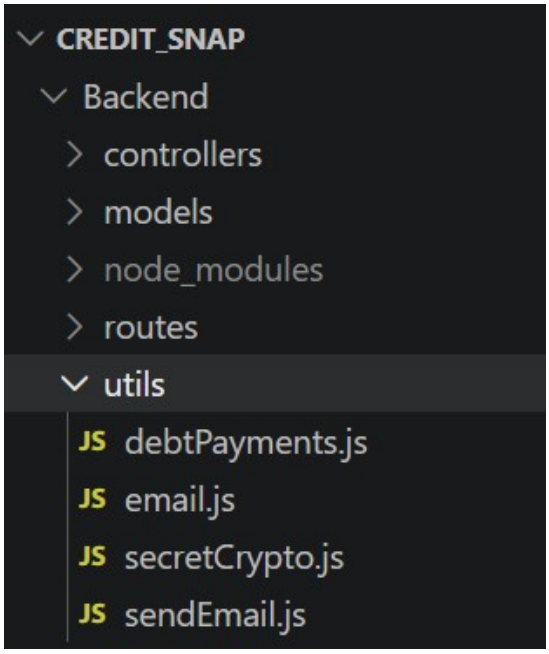
- **utils**/ – Helper Functions
  Contains reusable utility functions for common backend operations:
  - **debtPayments.js** – Utility functions for debt payment calculations and processing.
  - **email.js** – Helper functions for constructing and formatting emails.
  - **secretCrypto.js** – Handles encryption and cryptographic operations for secure data handling.
  - **sendEmail.js** – Manages the sending of emails for notifications, verification, and alerts.
- **Core Backend Files**
  - **app.js** – Initializes the Express application, sets up middleware, and connects to the MongoDB database.
  - **.env** – Stores environment variables such as database credentials, API keys, JWT secrets, and other sensitive configuration values.
  - **package.json** & **package-lock.json** – Define project dependencies and lock exact package versions for consistent installs.
  - **server.js** – The main entry point that starts the Express server and listens for incoming requests.


CREDIT_SNAP
Backend
controllers
models
node_modules
routes
utils
.env
app.js
package-lock.json
package.json
server.js


CREDIT_SNAP
Backend
controllers
analyticsController.js
canteenController.js
debtController.js
ordersController.js
paymentController.js
userController.js


CREDIT_SNAP
Backend
controllers
models
canteenModel.js
debtModel.js
menuItemModel.js
ordersModel.js
paymentModel.js
userModel.js


CREDIT_SNAP
Backend
controllers
models
node_modules
routes
analyticsRoutes.js
canteenRoutes.js
debtRoutes.js
ordersRoute.js
paymentRoutes.js
userRoutes.js


CREDIT_SNAP
Backend
controllers
models
node_modules
routes
utils
debtPayments.js
email.js
secretCrypto.js
sendEmail.js

3 Completeness

We were able to successfully implement nearly all of the features that we have mentioned in the SRS. We have listed the completed modules and functionalities in the sections below:

Implemented Features:

We have developed two separate interfaces in our system: one for students and one for canteen owners. Each interface is designed according to the specific needs of its users, ensuring smooth communication, efficient order handling, and proper financial tracking within the platform.

1. Authentication System:

The platform includes a secure and role-based authentication system for both students and canteen owners, ensuring security, preventing unauthorized access, and maintaining the integrity of user accounts within the platform:

- **Student Sign-up & Verification:** Students can create their accounts through a sign-up process using their email ID. During registration, a verification link is sent to the user's email to ensure that only valid and genuine IITK email accounts are created. The account is activated only after successful email verification.
- **Restricted Owner Registration:** For canteen owners, account creation is restricted and is manually handled by the development team. This ensures that only authorized canteens are registered on the platform and prevents misuse of the system.
- **Login & Password Recovery:** Both students and canteen owners can log in using their registered email and password. In case a user forgets their password, a "Forgot Password" option is provided. The system sends a secure password reset link to the registered email, allowing users to safely update their credentials.

2. Student Dashboard:

The student dashboard acts as the primary landing page and central hub for students, providing an immediate overview of their essential activities:

- **Structured Navigation:** A persistent sidebar provides seamless access to all major modules required by the SRS, including Home, Canteens (for browsing menus), View Debts, History, Help, and About Us.
- **Real-Time Debt Overview:** A prominent "Total Debt" card instantly displays the student's current accumulated outstanding balance, complete with a direct "Pay Now" button to facilitate quick settlements.
- **Live Order Tracking:** The "Current Orders" section provides a real-time feed of recent and active orders. It displays critical details at a glance, including the item name, canteen name, timestamp, price, and a dynamic status badge (e.g., "Accepted").

- **Alerts and Notifications:** A dedicated “Alerts” panel on the dashboard, alongside a notification bell in the top navigation bar, ensures students are immediately informed about order updates or payment reminders without needing to refresh the page.
- **Profile Management & Security:** Accessible via the user avatar in the top-right navigation bar, students can easily navigate to their profile settings to view and edit their personal details, as well as securely log out to end their session.

3. Canteen Owner Dashboard:

The canteen owner dashboard is designed to provide complete control over daily operations, featuring a structured interface tailored for efficient canteen management:

- **Live Active Orders & Canteen Status:** The Home section acts as a real-time feed for incoming student orders. Owners can instantly toggle their global “Canteen Status” (open/closed) and efficiently process, accept, reject, or clear active orders directly from the dashboard.
- **Menu Management:** Through the dedicated “Edit Menu” section, owners maintain full control over their offerings by adding new food items, editing existing details, deleting items, and toggling individual item availability.
- **Debt Tracking & Resolution:** The “Active Debts” tab allows owners to closely monitor each student’s outstanding balance, manually mark payments as completed (including offline cash payments), and send payment reminders.
- **Analytics:** An “Analytics” module provides valuable insights, displaying key metrics such as total earnings(through the website orders), order volume, and the most popular menu items to help owners make data-driven decisions.
- **History & Profile Management:** The dashboard includes a comprehensive “History” log for tracking all past transactions. Additionally, the top-right profile avatar allows owners to view and update their canteen account details and securely log out.

4. Credit Management System:

The platform implements a digital credit-based payment system that enables students to place food orders using a convenient “Pay Later” model. This system ensures controlled usage and accurate financial tracking through the following features:

- **Seamless Order Placement:** The checkout interface allows students to review their selected items, adjust quantities, and confirm their order request without upfront payment. The total cost is then automatically added to their outstanding balance for that specific canteen.
- **Canteen−wise Debt Tracking:** A dedicated “View debts” page provides a clear breakdown of accumulated dues for each canteen. It explicitly displays the student’s current debt against a predefined canteen credit limit (e.g., maximum limit of 3000 Rupees), preventing excessive debt accumulation.

- **Search & Sort Capabilities:** To easily manage multiple canteen tabs, the debt interface includes a targeted search bar to find specific canteens and a sorting dropdown to organize the debt list efficiently.
- **Flexible Payment Options:** Students can click the “Pay Debt” button next to any canteen’s tab to initiate direct online payments to settle their dues. Additionally, the system supports offline payments which the canteen owner can manually clear.
- **Owner-Side Verification:** Canteen operators have a corresponding debt management interface to monitor all active student tabs, verify incoming online payments, update offline payment statuses, and maintain a fully transparent ledger.

5. Order History & Ledger:

The platform provides a comprehensive history and ledger system for both students and canteen owners, ensuring transparency and efficient tracking of all past transactions:

- **Structured Student History:** Students can access their order history in a structured format that includes canteen name, ordered items with quantities, total cost, and precise timestamps. Each order is clearly labeled with its status (such as accepted or cancelled), allowing users to easily understand the outcome of their orders.
- **Advanced Search & Filtering:** To enhance usability, a search bar is integrated into the interface, enabling users to quickly locate specific orders based on item names or canteens. Additionally, advanced filtering options are provided, allowing users to refine results based on parameters such as order status, canteen name, date range, and amount range.
- **Sorting & Toggle Functionality:** Sorting functionality is also available, enabling users to organize their order history based on recent activity (newest or oldest first) or transaction amount (high to low or low to high). The interface also includes a toggle between order history and transaction history, allowing users to separately view order-related details and payment records.
- **Owner Transaction Ledger:** On the canteen owner side, a detailed transaction ledger is provided to track all completed orders along with customer details such as student name, contact information, and order data. Similar to the student interface, owners are also provided with search, filtering, and sorting functionalities to efficiently manage large volumes of transaction records. This system supports better financial tracking, simplifies reconciliation, and ensures accurate record maintenance for day-to-day operations.

6. Profile Management:

The system provides profile management features for both students and canteen owners. Users can view and update their personal details such as contact information and other relevant data. This ensures accurate communication and smooth functioning of the platform.

7. Help & Support Page:

A help section is included to assist users with common queries related to orders, payments, and system usage. It provides FAQs and guidance for resolving common issues, along with contact information for further support if required.

Future Improvements:

1. Mobile Application:

In the future, we plan to develop a dedicated mobile application for Android and iOS platforms. This will make the platform more accessible and convenient for students and canteen owners. The mobile app will also support push notifications, so users can receive instant updates about order status, payment reminders, and important alerts without needing to open the website.

2. Fake Account Detection:

To prevent misuse of the credit system, we aim to implement mechanisms for detecting fake or duplicate accounts. This can include identifying multiple accounts created using the same student credentials or suspicious usage patterns. Such accounts can be flagged or restricted to maintain fairness and avoid financial loss for canteen owners.

3. Rating and Review System:

We plan to introduce a rating and review system where students can provide feedback on canteens based on food quality, service, and delivery time. This will help maintain standards across canteens and allow new users to make better choices based on previous experiences shared by others.

4. Improved Payment Integration:

In future versions, we aim to enhance the payment system by integrating faster and more user-friendly options such as direct UPI payments. This will simplify the process of clearing dues and make transactions smoother for students.

5. Multi-Canteen Ordering:

Currently, orders are limited to a single canteen at a time. In future, we aim to allow students to place orders from multiple canteens in a single cart. This will improve user convenience and provide a more flexible ordering experience.

6. AI-Based Assistance:

In future, we plan to integrate AI-based assistance into the platform to improve user experience and efficiency. A simple chatbot can be added to help students with common queries such as checking their total debt, tracking orders, or searching for food items across canteens. For example, a student can ask, “In which canteens is Maggi available?” and the system can provide a quick and relevant response. Additionally, AI can be used to suggest popular food items based on user preferences and past orders. This feature will make the system more interactive and user-friendly while reducing manual effort for both students and canteen owners.

Appendix A - Group Log

The following is the group log detailing the implementation, integration, and deployment phases of the Credit Snap application.

Sl.No.	Date	Timings	Venue	Description
1	13/02/2026	18:00-19:30	RM Building	Divided implementation tasks based on the SDD timeline.
2	15/02/2026	18:00-19:30	RM Building	Status update.Completed frontend for signup and login pages and decided on a local-first development strategy to avoid early merge conflicts.
3	01/03/2026	18:00-20:00	RM Building	Divided the frontend and backend files among each group member.
4	04/03/2026	18:00-20:30	RM Building	Worked on the pages allocated to individual members.
5	08/03/2026	18:00-20:00	Hall 3	Decided folder structuring and some fixes on each other's pages
6	13/03/2026	22:00-23:30	RM Building	Created Github Repository. Discussed and started pushing the frontend files.
7	15/03/2026	17:00-20:00	RM Building	Discussed some fixes in each other's pages.
8	17/03/2026	18:00-21:00	RM Building	Discussed and started working on the backend files.
9	19/03/2026	09:00-11:00	Hall 3	Discussed about integration of frontend and backend
10	20/03/2026	21:00-23:00	RM Building	Started Integration of frontend and backend and fixing a few bugs.
11	23/03/2026	20:00-23:00	RM Building	Discussed and worked on some fixes in Integration.
12	24/03/2026	18:00-20:00	RM Building	Worked on some fixes in Integration and started the Implementation document .
13	26/03/2026	18:00-20:00	RM Building	Worked on the implementation document and tested some features.
14	27/03/2026	14:00-17:00	RM Building	Implemented the last adjustments to the documentation.